

WEB COMPONENTS I

Custom elements, plantillas HTML y Shadow DOM

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Miguel Ángel Muñoz

Contenido

1. Qué son los web components	3
2. Custom elements	3
2.1. Un primer custom element	3
2.2. El ciclo de vida	4
3. Plantillas HTML	4
4. Shadow DOM	5
5. Relación con los frameworks	5
6. Qué hemos aprendido	6

1. Qué son los web components

Cualquier persona que lleve un tiempo maquetando sitios web termina encontrándose con el mismo problema: hay piezas de interfaz —un botón, una tarjeta de perfil, un menú— que se repiten una y otra vez. Los **web components** nacen precisamente para resolver eso: son elementos HTML personalizados que se pueden importar y reutilizar en cualquier sitio web como si fueran una etiqueta más, por ejemplo `<user-card>`.

Al igual que ocurre con etiquetas nativas como `<button>` (que ya trae de fábrica un borde, un fondo gris y un color de letra), un web component puede llevar sus propios estilos predefinidos. La diferencia es que, si no se toman precauciones, esos estilos podrían chocar con los del resto de la página —o al revés—.

★ Lo más importante

- ▶ Un web component es un elemento HTML personalizado y reutilizable (su nombre siempre lleva un guion).
- ▶ Puede encapsular sus propios estilos y comportamiento sin afectar al resto de la página.
- ▶ Se puede importar y usar en cualquier sitio web sin copiar y pegar código.

2. Custom elements

Los **custom elements** son el conjunto de APIs de JavaScript que hacen posible crear estos elementos personalizados y definir su comportamiento. Como cualquier otra **API web**, no son más que construcciones que el propio navegador pone a disposición del desarrollador —igual que ocurre con la Fetch API, la Web Storage API o la API de geolocalización—.

El objeto encargado de todo esto es CustomElementRegistry, que expone varios métodos:

define() — registra un nuevo elemento personalizado.

get() — devuelve el constructor asociado a un nombre.

getName() — devuelve el nombre a partir del constructor.

upgrade() — fuerza la actualización de un elemento ya existente.

whenDefined() — devuelve una promesa que se resuelve cuando el elemento queda definido.

2.1. Un primer custom element

Definir un elemento personalizado exige crear una clase que extienda de HTMLElement y registrarla con un nombre que contenga un guion:

```
class MyButton extends HTMLElement {
  constructor() {
    super(); // ejecuta el constructor de HTMLElement
    this.attachShadow({ mode: 'open' });
    this.shadowRoot.innerHTML = `
    <style> button { margin: 50px; } </style>
    <button>Mi botón personalizado</button>
    `;
  }
}
customElements.define('my-button', MyButton);
```

A partir de aquí, usarlo es tan sencillo como escribir `<my-button></my-button>` en cualquier parte del HTML. Dentro del constructor, `this` siempre hace referencia a la instancia concreta del componente: si el elemento aparece varias veces en la misma página, cada copia mantiene su propio estado de forma completamente independiente.

2.2. El ciclo de vida

Un elemento personalizado no tiene por qué estar siempre presente en el DOM: se puede añadir o eliminar dinámicamente con JavaScript, y para reaccionar a esos momentos existen cuatro callbacks del ciclo de vida:

`connectedCallback` — se ejecuta cuando el elemento se inserta en el DOM por primera vez.

`disconnectedCallback` — se ejecuta cuando el elemento se elimina del DOM.

`adoptedCallback` — se ejecuta cuando el elemento se mueve a un nuevo documento.

`attributeChangedCallback` — se ejecuta cuando cambia uno de sus atributos observados.

★ Regla clave

- No metas toda la lógica del componente dentro del constructor: lo que va ahí se ejecuta en cuanto se importa el script, aunque el elemento todavía no esté en el DOM. La lógica que depende de que el componente ya esté insertado debe ir en `connectedCallback`.

3. Plantillas HTML

Las **plantillas HTML** ponen a disposición dos etiquetas pensadas para definir estructuras reutilizables: `<template>`, que declara un fragmento de HTML que no se renderiza hasta que se clona por JavaScript, y `<slot>`, que abre un hueco dentro del componente para que quien lo use pueda personalizar su contenido.

```
<template id="my-button">
<style> button { margin: 50px; } </style>
<button>
<slot name="button-text">Mi botón personalizado</slot>
</button>
</template>

<script>
const template = document.getElementById('my-button');
this.shadowRoot.appendChild(template.content.cloneNode(true));
</script>
```

El texto "Mi botón personalizado" actúa como contenido por defecto del slot; si alguien usa el componente así: `<my-button><span slot="button-text">Hola</span></my-button>`, ese contenido sustituirá al texto por defecto.

★ Lo más importante

- ▶ `<template>` define HTML reutilizable que no se pinta hasta que se clona.
- ▶ `<slot>` abre huecos personalizables dentro del componente, con contenido por defecto opcional.

4. Shadow DOM

El **Shadow DOM** es, literalmente, la versión "en la sombra" del DOM: un árbol de nodos oculto que se enlaza a un elemento concreto de la página y que permanece aislado del resto. Es precisamente lo que hace posible que un botón dentro de un web component no herede —ni contamine— los estilos globales del sitio donde se usa.

El Shadow DOM puede crearse en dos modos: **open**, en el que se puede acceder a él desde fuera a través de `elemento.shadowRoot`, o **closed**, en el que esa propiedad devuelve null y el árbol interno queda completamente oculto para el código externo.

El Shadow DOM no es exclusivo de los web components: cualquier elemento del DOM puede recibir uno mediante `elemento.attachShadow({ mode: 'open' })`, útil siempre que se quiera encapsular una parte concreta del HTML.

Conviene no confundirlo con el **Virtual DOM** que usan frameworks como React o Vue: este último no tiene nada que ver con el aislamiento de estilos, sino que es una copia del DOM guardada en memoria para poder agrupar varios cambios y aplicarlos de una sola vez, optimizando así la renderización.

★ Lo más importante

- ▶ El Shadow DOM aísla estilos y estructura de un componente respecto al resto de la página.
- ▶ Modo open: accesible desde fuera; modo closed: completamente oculto.
- ▶ El Virtual DOM (React, Vue) es un concepto distinto orientado a optimizar la renderización.

5. Relación con los frameworks

Es habitual haber oído hablar de web components siempre en el contexto de Angular, React, Vue o Polymer, lo que puede llevar a pensar que se trata de tecnologías distintas. No lo son: estos frameworks construyen sus propios componentes sobre exactamente la misma base nativa que hemos visto en este documento, añadiendo por encima una capa que simplifica su creación y aporta funcionalidades extra por defecto.

Entender primero los fundamentos nativos —antes que un framework concreto— es lo que permite, más adelante, comprender por qué cada framework toma las decisiones que toma, e incluso construir soluciones propias cuando un framework se queda corto.

6. Qué hemos aprendido

A lo largo de este documento hemos visto qué son los web components y las tres tecnologías que los sostienen: los custom elements (con su propio ciclo de vida), las plantillas HTML (template y slot) y el Shadow DOM, responsable de encapsular estilos y estructura.

Ninguna de estas piezas es exclusiva de los web components: se pueden aprovechar de forma independiente siempre que convenga encapsular o reutilizar una parte del HTML. La mejor forma de interiorizarlas es, como siempre, practicar creando componentes propios cada vez más completos.

¡Enhorabuena! Documento completado